Developer and administrator manual

Installation, configuration, deployment on exam.ajou.uz, administration, maintenance, backup and recovery, performance, security and troubleshooting

Version 1.3 · October 2026

Ajou University in Tashkent · AUT Secure Exam

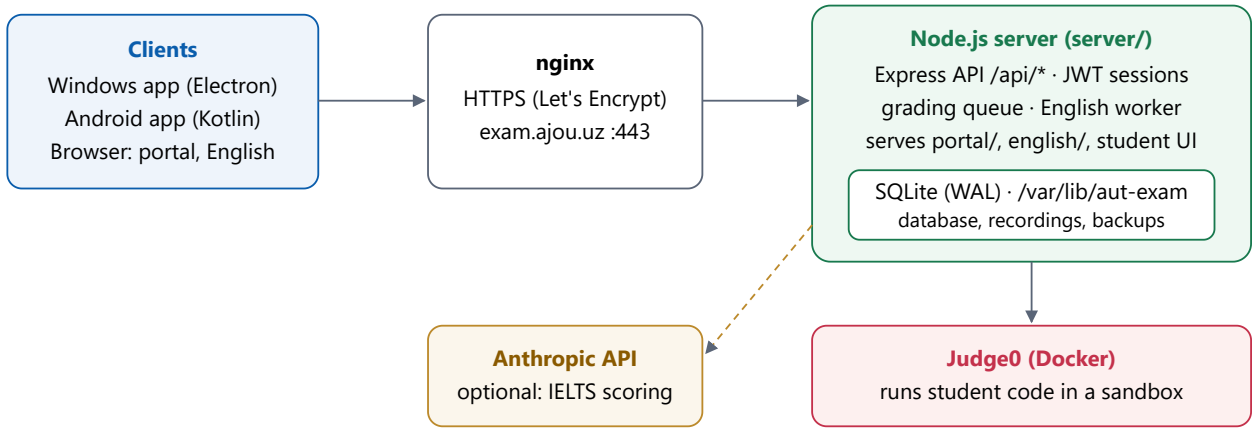
> For the developers and the IT staff who run the system. The measured capacity and all test evidence are in the separate **Project status and test report**; the step-by-step server installation is in **AUT_Secure_Exam_Deployment_Guide.pdf**.

Contents

1. System overview
2. Development setup
3. Configuration (.env)
4. Deployment on exam.ajou.uz
5. Roles and administration
6. Building and releasing the apps
7. Maintenance and monitoring
8. Backup and recovery
9. Performance and scaling
10. Security
11. Testing
12. Troubleshooting

1. System overview

One Node.js server holds everything: accounts, exams, answers, marks, English practice. All clients use its HTTPS API. Programming answers are run by Judge0 in a sandbox on a separate machine; the server never runs student code in production.



Components and how they talk to each other

Folder	
`server/`	API, database, grading, English engine, tests, load and crash tests
`portal/`	Professor and admin web portal
`english/`	English practice web app
`desktop/`	Windows app (Electron); renderer/ is also served at /student/
`android/`	Android app (Kotlin, Jetpack Compose)
`judge0/`	Judge0 sandbox (Docker Compose)
`deploy/`	Installer, nginx, systemd, backup timer, production .env
`docs/`	Guides and these manuals

2. Development setup

Requirements: Node.js 22 LTS, Git; Android Studio for the Android app; Docker + Linux (cgroup v1) for Judge0. Without Judge0, development uses `GRADER=local` (runs code on your PC, not a sandbox) or `GRADER=off`.

```
# Windows or Linux, Node.js 22 LTS
git clone https://github.com/ramzanali-blip/AJOU-EXAM.git
cd AJOU-EXAM/server
npm ci
copy .env.example .env          (Linux: cp .env.example .env)
npm run seed                    # demo accounts, development only
npm start                       # http://localhost:4000
npm test                        # automated tests
```

3. Configuration (.env)

Setting	Example	Meaning
`NODE_ENV`	`production`	Refuses weak settings (short JWT_SECRET, GRADER=local).
`JWT_SECRET`	`96 hex chars`	Signs sessions. Generated by the installer. Changing it signs everyone out.
`JWT_EXPIRES_IN`	`12h`	Session length.
`DB_PATH`	`/var/lib/aut-exam/exam.db`	SQLite database file.
`PROFESSOR_SIGNUP_CODE`	`(empty)`	Empty = professors created by an administrator only.
`GRADER`	`judge0 | off`	judge0 = sandboxed code marking; off = code marked by hand (public test servers).
`JUDGE0_URL / JUDGE0_AUTH_TOKEN`	`http://10.x.x.x:2358`	Judge0 address (private network) and token.
`GRADER_CONCURRENCY`	`4`	Attempts graded at the same time.
`AUTH_RATE_LIMIT`	`50`	Sign-ins per IP per 15 min. Raise it (e.g. 1000) when a whole class shares one campus IP.
`UV_THREADPOOL_SIZE`	`8 (systemd)`	Worker threads for password hashing at sign-in rushes.
`ANTHROPIC_API_KEY`	`(optional)`	AI examiner for English speaking/writing.
`ENGLISH_ADMIN_EMAILS`	`dev@ajou.uz`	Library editors for English tasks.
`ENGLISH_CONCURRENCY`	`2`	Speaking answers transcribed at once (CPU-heavy).

4. Deployment on exam.ajou.uz

The university website www.ajou.uz is not changed: the system gets its own sub-domain. Full step-by-step instructions, server sizes and the go-live checklist: **AUT_Secure_Exam_Deployment_Guide.pdf**. In short:

```
# 1. DNS (university IT): A record  exam ->
# 2. On the Ubuntu 22.04/24.04 server, from the source folder:
sudo DOMAIN=exam.ajou.uz ADMIN_EMAIL=it@ajou.uz bash deploy/install-ubuntu.sh
# 3. Judge0 (see judge0/README.md), then in /opt/aut-exam/server/.env:
#    JUDGE0_URL=http://:2358   JUDGE0_AUTH_TOKEN=
sudo aut-exam restart
# 4. First administrator (a professor account with admin rights)
sudo aut-exam create-professor "Full Name" name@ajou.uz 'strong-password' --admin
sudo aut-exam health
```

5. Roles and administration

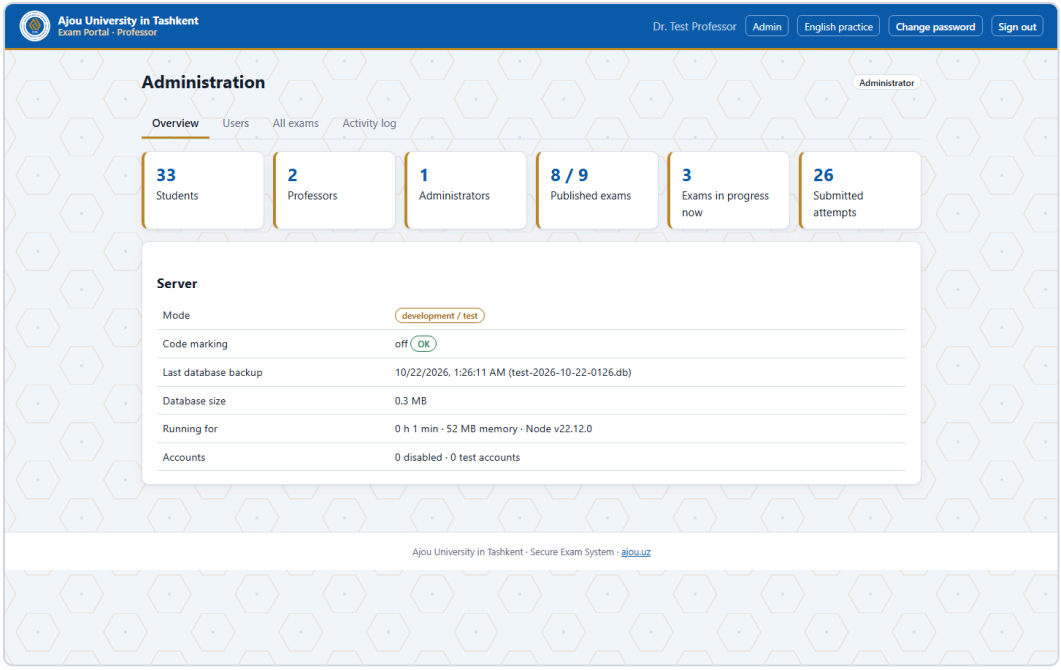
There are three roles. Administrators are professor accounts with administrator rights. The server checks the role and ownership on every request (a professor sees only their own exams; a student only their own attempts).

Action	Student	Professor	Admin
Sign in, join exams, take exams, see released results, English practice	✓	✓	✓
Create and schedule exams, questions, enrol students, live monitor, marking, results	–	✓	✓
Reset passwords of students enrolled in own exams	–	✓	✓
See and manage all accounts (disable, reset password, change role, admin rights)	–	–	✓
See all exams, activity log, server status and last backup	–	–	✓

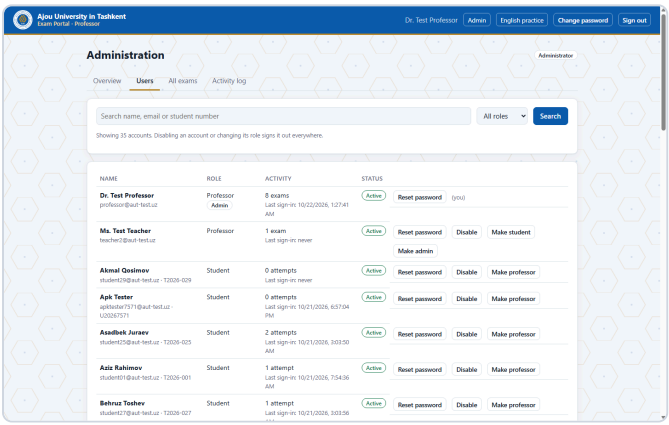
Create accounts and administrators on the server:

```
sudo aut-exam create-professor "Full Name" name@ajou.uz 'password'          # professor
sudo aut-exam create-professor "Full Name" name@ajou.uz 'password' --admin  # administrator
cd /opt/aut-exam/server && sudo -u aut-exam npm run make-admin -- name@ajou.uz [--remove]
sudo aut-exam remove-test-accounts --yes      # before real exams
```

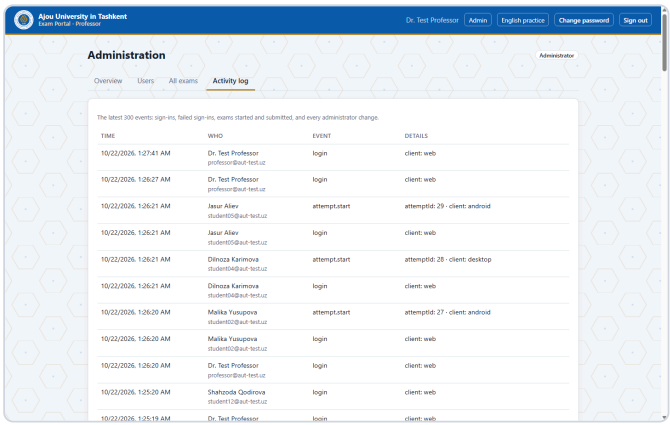
In the portal, administrators see **Admin** in the top bar: server status and last backup, all users (search, disable, reset password, change role, admin rights), all exams, and the activity log of sign-ins, failed sign-ins and every administrator change.



Admin → Overview: users, exams in progress, code marking, last backup



Admin → Users



Admin → Activity log

6. Building and releasing the apps

```
cd android
gradlew assembleStandardRelease bundleStandardRelease     # signed APK/AAB for https://exam.ajou.uz
gradlew assembleStandardRelease -PserverUrl=https://other.ajou.uz
gradlew assembleTrialDebug -PtestServerUrl=http://:4200   # "AUT Exam TEST"
cd desktop
npm ci && npm run dist                                     # Windows installer (examServerUrl in
package.json)
```

Release keys: `android/aut-release.jks` + `android/keystore.properties` (never in git; back them up). Raise `versionCode` / `versionName` for every Android release and `version` in desktop/package.json.

7. Maintenance and monitoring

The service restarts automatically after a crash or reboot. Check the Admin overview daily during exam periods.

```
sudo aut-exam status      # service state
sudo aut-exam logs        # live log (journald)
sudo aut-exam health      # {"ok":true,"grader":"judge0",...}
sudo aut-exam restart     # graceful: requests in flight are finished
sudo aut-exam backup      # backup now
# update: copy the new source, then run the installer again (data and .env are kept)
sudo DOMAIN=exam.ajou.uz ADMIN_EMAIL=it@ajou.uz bash deploy/install-ubuntu.sh
```

8. Backup and recovery

Automatic: every hour (systemd timer), the newest 60 copies in `/var/lib/aut-exam/backups`, made with SQLite `VACUUM INTO`, which is safe while students are writing. Copy them to another machine every night:

```
# /etc/cron.d/aut-exam-offsite: copy backups and recordings to another machine every night at 02:30
30 2 * * * root rsync -a /var/lib/aut-exam/backups /var/lib/aut-exam/english-audio backup@backup-
host:/srv/aut-exam/
```

Restore (tested by `npm run crash-test`):

```
sudo systemctl stop aut-exam
sudo cp /var/lib/aut-exam/backups/exam-YYYY-MM-DD-HHMM.db /var/lib/aut-exam/exam.db
sudo rm -f /var/lib/aut-exam/exam.db-wal /var/lib/aut-exam/exam.db-shm
sudo chown aut-exam:aut-exam /var/lib/aut-exam/exam.db
sudo systemctl start aut-exam && sudo aut-exam health
```

> Test a restore on a copy every month. A backup on the same disk does not survive a disk failure.

9. Performance and scaling

Design: one Node.js process; SQLite in WAL mode (each answer save is one short transaction); password hashing (scrypt) on worker threads; grading in a background queue; sessions are stateless tokens. Measured results are in the test report. Levers:

Bottleneck	What to do
Sign-in rush	Students sign in 10–15 minutes before the exam (sessions last 12 h). More CPU cores + UV_THREADPOOL_SIZE.
Code marking	More Judge0 workers / a bigger Judge0 VM; GRADER_CONCURRENCY.
English speaking	ENGLISH_CONCURRENCY = 1–2 per CPU core; more cores.
Server size	Up to ~1,000 simultaneous students: 4 vCPU / 8 GB; more: 8 vCPU / 16 GB (verify with load-test on the server).
Beyond one server	Thousands of simultaneous students across many exams: move to PostgreSQL and several server instances behind nginx (schema in server/src/db.js maps directly).

10. Security

Area	How it is protected
Passwords	scrypt (N=16384), old bcrypt hashes upgraded at sign-in; minimum 8 characters.
Sign-in protection	10 wrong passwords lock an account for 15 minutes; per-IP limit (AUTH_RATE_LIMIT).
Sessions	Signed tokens (HS256, 12 h). Password change/reset, disabling or a role change ends all sessions at once.
Roles	Checked on every request; ownership checks on exams and attempts; administrators cannot remove their own rights.
Transport	HTTPS only (nginx + Let's Encrypt), HSTS; release Android app refuses http.
Exam integrity	Server-side timer and deadline; lockdown and event reporting in the apps; answers locked after submit; answer keys never sent to students.
Code execution	Only in Judge0 (separate VM, private network); GRADER=local is refused in production.
Secrets	server/.env (640, root:aut-exam), judge0.conf, Android key: never in git or e-mail.
Audit	Sign-ins, failed sign-ins, exam start/submit and every administrator change are logged.
Dependencies	npm audit: 0 known vulnerabilities (October 2026). Run it before every release.

☐ HTTPS works, http redirects
☐ NODE_ENV=production, PROFESSOR_TEST_SIGNUP_CODE empty
☐ Judge0 port reachable only from the exam server
☐ SSH by key only; unattended-upgrades on
☐ Test accounts removed; default/demo accounts deleted
☐ Backups copied off the server; restore tested

11. Testing

All test tools start their own throw-away server and database; real data is never touched.

```
cd server
npm test                          # 14 automated tests: exam flow, grading, English, security,
GRADER=off
npm run load-test                 # stages 100..2000 students (own throw-away server)
npm run load-test -- 500 1000     # chosen stages; PRELOGIN=1 = students signed in before the exam
npm run crash-test                # kills the server mid-exam, restarts, checks every answer; backup
+ restore
LOAD_URL=https://exam.ajou.uz LOAD_PROF_EMAIL=... LOAD_PROF_PASSWORD=... npm run load-test -- 200
# real server
```

Public test server from a PC (no account needed): `Start Public Test Server.cmd` starts the test server (`server/.env.test`, port 4200, `GRADER=off`) and a Cloudflare quick tunnel, and shows the link to share. The download page on that link connects the TEST app with one tap.

12. Troubleshooting

Problem	Check / fix
502 Bad Gateway	sudo aut-exam logs; often a typo in .env or a too-short JWT_SECRET.
Service does not start after an update	node -v must be 22+; run the installer again; check journal.
"Grader error" / Judge0 unreachable	docker compose ps on the Judge0 VM; JUDGE0_URL/token; firewall; cgroup v1. Then Re-run auto-grading.
Students cannot sign in, many at once	Look for 429 in the log: raise AUTH_RATE_LIMIT if a class shares one IP.
"Account locked" for a student	Wait 15 minutes, or reset the password (Admin → Users).
Slow at exam start	Ask students to sign in early; check CPU (top); add cores; see section 9.
Database is locked / disk full	df -h; free space; old backups are pruned automatically (60 kept).
Certificate expired	sudo certbot renew; check that port 80 is open.